

Application Architecture: ALB, Okta and ECS in a VPC

How the application is hosted, how users sign in, and what each component does · September 2026

1. Overview

The application runs entirely inside a VPC. A single Application Load Balancer (ALB) is the only public entry point. The ALB signs users in with **Okta**, then routes each request by path:

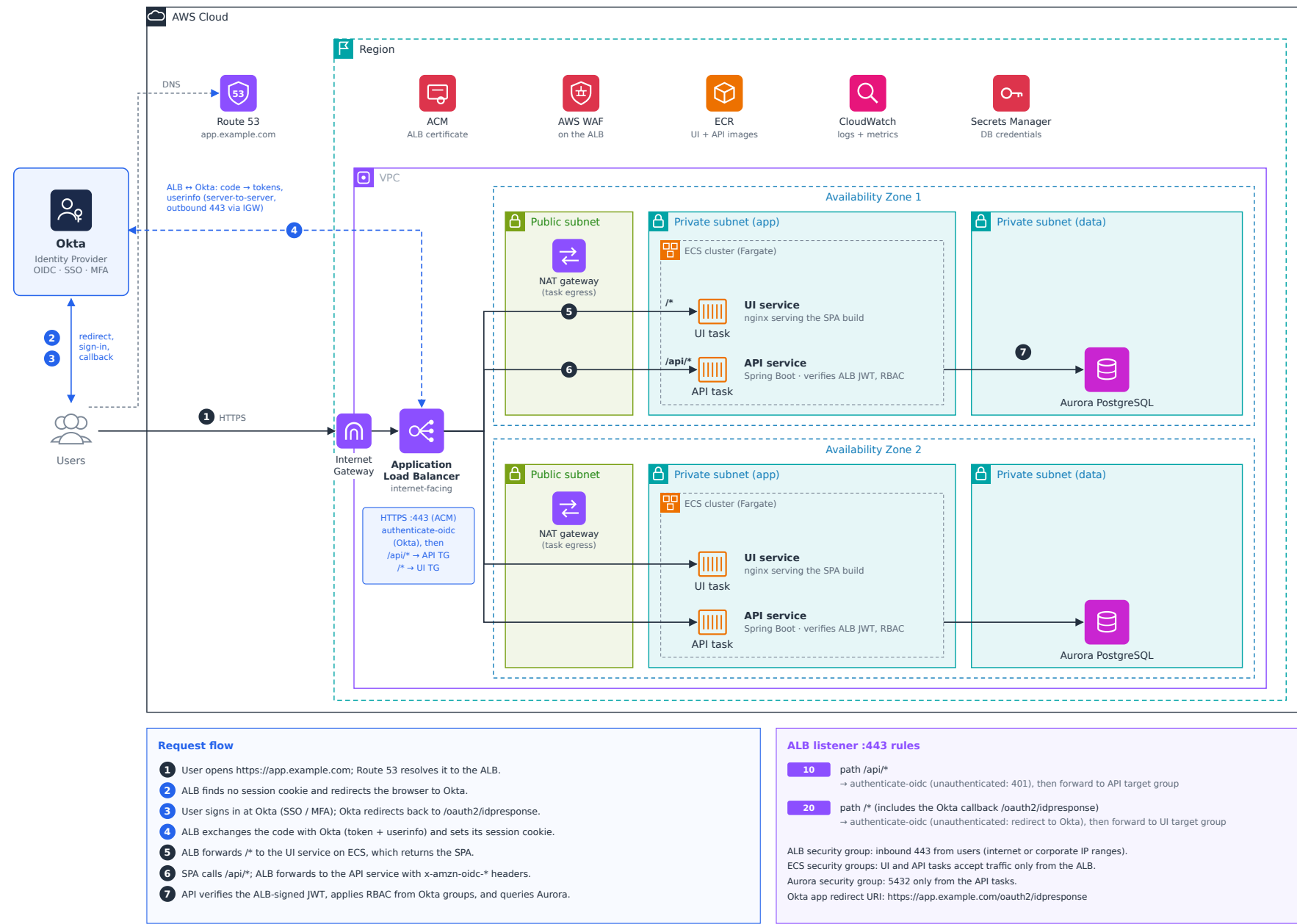
- `/api/*` goes to the **API service**, a Spring Boot application.
- Everything else (`/*`) goes to the **UI service**, which is nginx serving the single-page app (SPA) build.

Both services run as containers on **Amazon ECS with Fargate**, in private subnets across two Availability Zones. The API stores data in **Amazon Aurora PostgreSQL**, which sits in its own private data subnets.

Design principles

- **One entry point.** All traffic enters through the ALB. No compute or data resource is reachable from the internet.
- **Login at the edge, permissions in the service.** The ALB checks who the user is (authentication) with Okta. The API decides what the user may do (authorization), based on the user's Okta groups.
- **High availability by default.** Every tier runs in two Availability Zones.
- **Least privilege on the network.** Each tier accepts traffic only from the tier directly in front of it.

2. Architecture diagram



3. Request flow

#	STEP	WHERE IT HAPPENS
1	The user opens <code>https://app.example.com</code> . Route 53 resolves the name to the ALB.	Browser → Route 53 → ALB
2	The ALB finds no session cookie and redirects the browser to Okta's sign-in page.	ALB → browser → Okta
3	The user signs in at Okta (SSO / MFA). Okta redirects the browser back to <code>/oauth2/idpresponse</code> with a one-time authorization code.	Okta → browser → ALB
4	The ALB exchanges the code for tokens at Okta's token endpoint, reads the user's profile and groups from the userinfo endpoint, and sets its own session cookie.	ALB → Okta (server-to-server)
5	The ALB forwards <code>/*</code> requests to the UI service, which returns the SPA.	ALB → UI tasks
6	The SPA calls <code>/api/*</code> . The ALB forwards these requests to the API service and adds <code>x-amzn-oidc-*</code> headers describing the user.	ALB → API tasks
7	The API verifies the ALB-signed token, applies role-based access (RBAC) from the Okta groups, and queries Aurora.	API → Aurora

Steps 2–4 happen only when the user has no valid session. After that, every request is authenticated by the ALB's session cookie alone. The ALB does not call Okta again until the session expires.

4. Components

4.1 Users and DNS

Amazon Route 53

- **What it is:** AWS's managed DNS service.
- **Role here:** resolves `app.example.com` to the ALB.
- **Key configuration:**
 - Use an **alias A record** pointing to the ALB. An alias record follows the ALB's changing IP addresses automatically and has no query charge.
 - Optionally, add a health check on the record.

4.2 Identity

Okta (identity provider)

- **What it is:** Okta is the organisation's identity provider. It holds user accounts and groups, and handles sign-in, SSO and MFA.
- **Role here:** the ALB uses Okta as its OpenID Connect (OIDC) provider.
- **Key configuration:**
 - **OIDC Web Application.** Use the Authorization Code grant, with redirect URI `https://app.example.com/oauth2/idpresponse` .
 - **Groups.** Assign the groups that should have access, e.g. `app-readonly` and `app-editor` . Users outside the assigned groups are blocked at Okta.

- **groups claim.** Add it on the authorization server, with a filter such as *Starts with* `app-`. This is what the API uses for RBAC.

4.3 Entry point

Internet Gateway (IGW)

- **What it is:** the VPC's connection to the internet.
- **Traffic it carries:**
 - **Inbound:** users' requests to the ALB.
 - **Outbound:** the ALB's own calls to Okta's token and userinfo endpoints (step 4 of the flow).

Application Load Balancer (ALB)

- **What it is:** the only public entry point to the application. It's internet-facing and deployed in the public subnets of both Availability Zones.
- **What it does:**
 - **Ends HTTPS.** The HTTPS connection terminates here, using an ACM certificate.
 - **Signs users in with Okta.** It uses its built-in `authenticate-oidc` action.
 - **Routes by path.** Requests go to one of two target groups depending on the URL.

Listener rules (HTTPS :443)

PRIORITY	CONDITION	ACTIONS
10	Path <code>/api/*</code>	<code>authenticate-oidc</code> (unauthenticated: deny , returns 401) → forward to API target group
20	Path <code>/*</code>	<code>authenticate-oidc</code> (unauthenticated: authenticate , redirects to Okta) → forward to UI target group
HTTP :80	Any	Redirect to HTTPS

- **Why `/api/*` returns 401 instead of redirecting:** background API calls from the browser can't follow a redirect to Okta's login page. So if the session expires, the API returns 401 and the SPA reloads the page, which goes through the normal login redirect.
- **Okta callback:** `/oauth2/idpresponse` matches rule 20, so the ALB handles the callback with no extra rule.

Other settings

- **Target groups:** two, one per service (UI and API). Both use target type `ip`, as Fargate requires.
- **Health checks:** go directly to the targets, not through the listener rules, so they don't need a login. Use `/healthz` on the UI and `/actuator/health` on the API.
- **Session timeout:** set to match your security policy, e.g. 1 hour; the ALB default is 7 days. Users removed from an Okta group keep their access until their ALB session expires.

AWS Certificate Manager (ACM)

- Provides the TLS certificate for `app.example.com` on the ALB's HTTPS listener.
- The certificate must be issued in the **same region as the ALB**.
- ACM renews DNS-validated certificates automatically.

AWS WAF

- A web application firewall attached to the ALB. It filters requests before they reach the listener rules.

- **Recommended rules:**

- AWS managed rule groups: Core rule set and Known bad inputs.
- A rate-based rule to limit abusive clients.
- Optionally, an IP allow-list if the app is internal only.

4.4 Network

VPC and Availability Zones

- **What it is:** a private network for the application.
- **Layout:** two Availability Zones (AZs), each with three subnet tiers. If one AZ fails, the ALB, ECS and Aurora keep serving from the other.

SUBNET TIER	CONTAINS	ROUTE TO INTERNET
Public	ALB nodes, NAT gateway	Direct, via the Internet Gateway
Private (app)	ECS tasks: UI and API	Outbound only, via the NAT gateway
Private (data)	Aurora instances	None

NAT gateway

- **What it does:** lets tasks in the private app subnets make **outbound** connections, while nothing on the internet can connect in. Tasks use it to pull images from ECR, send logs and metrics to CloudWatch, read secrets, and call external APIs.
- **One per AZ:** this keeps each AZ independent if the other fails.
- **Cheaper alternative:** VPC endpoints can replace most NAT traffic. You'd need endpoints for:
 - ECR (`ecr.api` and `ecr.dkr`), plus an **S3 gateway endpoint**, because ECR stores image layers in S3
 - CloudWatch Logs
 - Secrets Manager

Security groups

Each tier accepts traffic only from the tier directly in front of it:

SECURITY GROUP	INBOUND	SOURCE
ALB	443 (and 80 for the redirect)	Users: <code>0.0.0.0/0</code> , or corporate IP ranges for an internal app
UI tasks	Container port (e.g. 8080)	ALB security group only
API tasks	Container port (e.g. 8080)	ALB security group only
Aurora	5432	API task security group only

The ALB's security group also needs **outbound 443 to the internet**, so the ALB can reach Okta. Without it, users can sign in at Okta but get a 500 error on the redirect back.

4.5 Compute

Amazon ECS cluster (Fargate)

- **What it is:** runs the UI and API as containers on Fargate, so there are no servers to manage or patch.

- **Networking:** each task gets its own network interface and security group (`awsvpc` mode).
- **Services:** each service keeps at least one task in each AZ and registers its tasks with its ALB target group.
- **IAM roles:** each task definition has two roles:
 - **Task execution role:** used by ECS itself to pull the image from ECR, write logs to CloudWatch, and read secrets for the container.
 - **Task role:** used by the application code at runtime. Grant only what the app itself calls.
- **Scaling:** use Application Auto Scaling with target tracking, on CPU utilisation or `ALBRequestCountPerTarget`.
- **Deployments:** use rolling updates with the deployment circuit breaker, so a failing release rolls back automatically.

UI service

- **What it runs:** nginx serving the compiled SPA (HTML, JS, CSS).
- **Key configuration:**
 - **Client-side routes:** nginx sends any unknown path back to `index.html`, so the SPA's own routing still works when a user refreshes a page.
 - **Caching:** give hashed asset files long-lived `Cache-Control` headers so browsers cache them; `index.html` should be `no-cache`.
 - **No application logic:** the UI never talks to the database directly.

API service

- **What it runs:** the Spring Boot backend.
- **Checking the user on each request:**
 1. **Verify the ALB token.** Check the `x-amzn-oidc-data` header: the signature (ES256), that the `signer` is this ALB's ARN, and that it hasn't expired.
 2. **Map groups to roles.** Turn the Okta `groups` claim into application roles, e.g. `app-readonly` → `READER` and `app-editor` → `EDITOR`.
 3. **Enforce permissions.** Check each endpoint's rules with `@PreAuthorize`.
- **Database access:** connects to Aurora with credentials read from Secrets Manager.

4.6 Data

Amazon Aurora PostgreSQL

- **What it is:** the application's relational database, running in the private data subnets.
- **Layout:** a **writer** instance in one AZ and a **reader replica** in the other. If the writer fails, Aurora promotes the replica automatically, typically within about a minute.
- **Key configuration:**
 - **Network:** reachable only from the API tasks, on port 5432.
 - **Encryption:** at rest with KMS, and TLS in transit.
 - **Backups:** automated backups with a retention period suited to your recovery point objective.
 - **Credentials:** the master password is managed in Secrets Manager.
- **Scaling reads:** point read-heavy queries at the reader endpoint.

4.7 Supporting services

Amazon ECR

- **What it is:** the private container image registry for the UI and API images.

- **Key configuration:**
 - **Scan on push:** checks new images for vulnerabilities (CVEs).
 - **Immutable tags:** a deployed tag can't be overwritten.
 - **Lifecycle policy:** removes old images automatically.

Amazon CloudWatch

- **What it does:** central logs, metrics and alarms.
- **Logs:** ECS tasks send container logs with the `awslogs` driver. Container Insights adds task-level CPU and memory metrics.
- **Alarms worth setting:**
 - ALB `HTTPCode_Target_5XX_Count`
 - ALB `TargetResponseTime`
 - `UnHealthyHostCount` per target group
 - Aurora CPU and free memory

AWS Secrets Manager

- **What it stores:** secrets such as the Aurora credentials.
- **How ECS uses it:**
 - ECS injects secrets into containers as environment variables, via the task definition's `secrets` block.
 - The task execution role needs `secretsmanager:GetSecretValue` for those secrets.
 - Rotation can be enabled for the database password.
- **Okta client secret:** this lives in the ALB's `authenticate-oidc` settings, not in Secrets Manager. Keep it in your infrastructure-as-code secret store, and pass it to the ALB when you deploy.

5. Security model

CONCERN	HOW IT'S HANDLED
Who is the user? (authentication)	ALB + Okta (OIDC). No request reaches the UI or API without a valid ALB session.
What may they do? (authorization)	The API maps Okta groups to roles and enforces them per endpoint.
Can the identity headers be faked?	The API checks the ALB token's signature and its <code>signer</code> , and only the ALB's security group can reach the tasks.
Network exposure	Only the ALB is public. The app and data subnets have no inbound route from the internet.
Data protection	TLS from browser to ALB, and between the API and Aurora. Aurora is encrypted at rest.
Secrets	Held in Secrets Manager and injected at runtime, never baked into images.
Common web attacks and abuse	AWS WAF on the ALB (managed rules and rate limiting).

6. Availability and scaling

TIER	HOW IT'S RESILIENT	HOW IT SCALES
ALB	Nodes in both AZs, managed by AWS	Scales automatically
UI / API	At least one task per AZ; unhealthy tasks are replaced	ECS service auto scaling
NAT	One gateway per AZ	Managed by AWS
Aurora	Writer + replica in separate AZs; automatic failover	Add reader replicas; scale instance size

7. Implementation checklist

1. **Okta:** create the OIDC Web App with redirect URI `https://app.example.com/oauth2/idpresponse`, then assign groups and add the `groups` claim.
2. **ACM:** issue the certificate for `app.example.com` in the ALB's region.
3. **ALB:**
 - Create the HTTPS listener with both rules (Section 4.3) and the HTTP → HTTPS redirect.
 - Set the session timeout.
 - Allow outbound 443 in the ALB security group so it can reach Okta.
4. **ECS:**
 - Create both services, with their target groups and health check paths.
 - Set up the security groups as in Section 4.4.
 - Add the task execution and task roles.
5. **API:** implement ALB token verification (signature, signer, expiry) and the group-to-role mapping.
6. **UI:** configure nginx to send unknown paths to `index.html` and set cache headers. The SPA should reload the page when an API call returns 401.
7. **Operations:**
 - Attach WAF to the ALB.
 - Set up the CloudWatch alarms.
 - Configure ECR scanning and lifecycle rules.
 - Store the Aurora credentials in Secrets Manager.
 - Optionally enable ALB access logs. This is the only part that needs an S3 bucket.